



Optimizing parameters in swarm intelligence using reinforcement learning: An application of Proximal Policy Optimization to the iSOMA algorithm

Lukáš Klein^{a,b,*}, Ivan Zelinka^a, David Seidl^{a,b}

^a Department of Computer Science, FEI, VSB - Technical University of Ostrava, Ostrava, 70800, Czech Republic

^b ENET Centre, VSB - Technical University of Ostrava, Ostrava, 70800, Czech Republic

ARTICLE INFO

Keywords:

Self-Organizing Migrating Algorithm
Optimization algorithm
Swarm intelligence
Numerical optimization
Reinforcement learning

ABSTRACT

This paper presents a new algorithm for optimizing parameters in swarm algorithm using reinforcement learning. The algorithm, called iSOMA-RL, is based on the iSOMA algorithm, a population-based optimization algorithm that mimics the competition-cooperation behavior of creatures to find the optimal solution. By using reinforcement learning, iSOMA-RL can dynamically and continuously optimize parameters, which can play a crucial role in determining the performance of the algorithm but are often difficult to determine. The reinforcement learning technique used is the state-of-the-art Proximal Policy Optimization (PPO), which has been successful in many areas. The algorithm was compared to the original iSOMA algorithm and other algorithms from the SOMA family, showing better performance with only constant increase in computational complexity depending on number of function evaluations. Also we examine different sets of parameters to optimize and different reward functions. We also did comparison to widely used and state-of-the-art algorithms to illustrate improvement in performance over the original iSOMA algorithm.

1. Introduction

Optimization problems are becoming increasingly prevalent with the continuous development of science and technology in our world [1]. These problems consists of finding global optimal solution. Swarm Intelligence (SI) methods, such as Differential Evolution (DE) [2], Ant colony optimization [3], Artificial Bee Colony (ABC) [4], Particle Swarm Optimization (PSO) [5], and Self-Organizing Migrating Algorithm (SOMA) [6,7], have been shown to be effective in solving such problems. SOMA, which was proposed in the 2000s, is a population-based optimization algorithm that mimics the competition-cooperation behavior of creatures to find the optimal solution. SOMA is our main interest in this paper. SOMA has many variants such as self-adapting SOMA [8], clustering SOMA-CLP [9], non linear NL-SOMA-CLP [10], SOMAQI (with Quadratic Interpolation) [11] and many others.

The iSOMA algorithm [12] is the one of the latest variants of SOMA and is a swarm-based optimization algorithm that targets weakness of SOMA algorithms, which are stopping criteria, move on the edge and search for nonsense. iSOMA divides the algorithmic framework into four processes, each of which improves: the initialization process, self-organizing process, migrating process, and replacement process. The initialization process uses uniformly distributed random numbers to

start the algorithm. The self-organizing process selects the best individuals in a group to move towards the best individual in another group. The migrating process changes the order of jumps and immediately updates the solutions, narrowing the search space. The replacement process is taken when the algorithm cannot find a better global optimal solution. All these processes have parameters defining behavior of the algorithm.

The iSOMA algorithm has demonstrated good performance compared to other well-known algorithms such as PSO, GA, and DE. It defeated the T3 A version [13], which was ranked 3rd in the CEC19 competition [14], and also the Pareto-based SOMA [15], which was ranked 5th in the CEC19 competition [16]. The competition was the 100-Digit Challenge, which was reported in [17]. Despite its success, one of the limitations of the iSOMA algorithm is the difficulty in optimizing algorithm's parameters. For example, one parameter is scaling based on function evaluations (FEs) in ratio to max FEs, and others should be set up depending on the presence of local traps. These parameters can play a crucial role in determining the performance of the algorithm, yet their optimal values are often difficult to determine and can change depending on the problem at hand or even within a single problem. Also hyper tuning of these parameters can be costly and search space is also large and optimal parameters can change while optimizing single function.

* Corresponding author at: Department of Computer Science, FEI, VSB - Technical University of Ostrava, Ostrava, 70800, Czech Republic.
E-mail address: kle0133@vsb.cz (L. Klein).

This is where our proposed new algorithm, which we call iSOMA-RL, comes in. By using reinforcement learning, which is pretty old technique [18], we can dynamically and continuously optimize these algorithm's parameters. We choose recent and state-of-the-art Proximal Policy Optimization (PPO) [19], which has success in many areas [20–22]. This is a novel application of PPO for function optimization and we continuously optimize the variables, in contrast with other authors using reinforcement learning [23,24], where their action space was discrete. Also most researchers used q-learning or other older RL algorithm [25,26]. We examined three variants of the algorithm, each of which optimizes different set of algorithm's parameters. We also compared the performance of iSOMA-RL to the original iSOMA algorithm and thus could be compared to other algorithms from the SOMA family.

Our research objectives and the benefits of this study can be summarized as follows:

1. Overcoming a limitation of iSOMA in selecting parameters, which can impact the algorithm's performance.
2. Utilizing Proximal Policy Optimization (PPO) to dynamically and continuously optimize these parameters.
3. Introducing a novel application of PPO for function optimization, distinguishing it from previous discrete action space reinforcement learning methods used by other authors.
4. Examining three variants of iSOMA-RL, each optimizing different sets of parameters.
5. Investigating different reward functions to determine if a specifically tailored reward function is necessary for PPO enhancement.
6. This method can be applied to any other algorithm with parameters such as PSO and others.

Our proposed new algorithm, iSOMA-RL, addresses this limitation by using reinforcement learning to dynamically and continuously optimize these algorithm's parameters. Reinforcement learning, which is old idea [18]. We chose to use the state-of-the-art Proximal Policy Optimization (PPO) [19], which has been successful in many areas [20–22]. This is a novel application of PPO for hybridization of function optimization algorithm and we continuously optimize the parameters as action space is continuous, in contrast with other authors who have used reinforcement learning for function optimization [23,24], where the action space was discrete. Additionally, most researchers have used q-learning or other older RL algorithms [25–27]. We examined three variants of the algorithm, each of which optimizes a different set of algorithm's parameters. In addition, we will examine the rewards function. We also compared the performance of iSOMA-RL to the original iSOMA algorithm and other algorithms from the SOMA family.

This article is divided into several parts. In the first section, we provide background information on the iSOMA algorithm, current limitations, and an overview of reinforcement learning. In the second and third section, we describe our proposed methods for optimizing algorithm's parameters and applying reinforcement learning to the iSOMA algorithm. The fourth section presents the results of our study, including a comparison of our proposed methods to the original iSOMA algorithm, an evaluation of the effectiveness of our methods, and a discussion of the results and their implications. Finally, in the fifth section, we summarize our findings and discuss future research directions.

2. iSOMA background and its parameters

This section will first introduce the original SOMA algorithm, followed by an explanation of the improvements made in the iSOMA algorithm and a comparison to other algorithms.

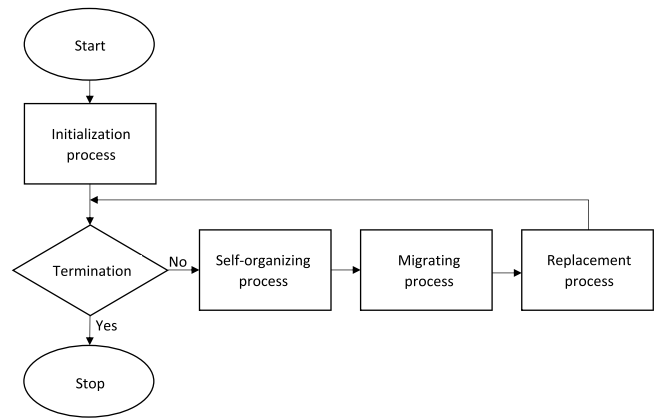


Fig. 1. The flowchart of the iSOMA from [12].

2.1. Original SOMA

The Self-Organizing Migrating Algorithm [7,28] is a swarm-based intelligence optimization algorithm that finds an optimal solution to a given problem by interactions between individuals in the population according to a given rule. SOMA starts by generating a population of individuals as candidate solutions to a given problem with a set of boundaries for each variable. These individuals are then evaluated by a fitness function and enter the first migration loop. In each migration loop, the individual with the lowest fitness value is chosen as the leader, and the remaining individuals are migrants who move towards the leader. This movement is performed in small steps and with a certain level of randomness to maintain population diversity. The best position in the moving trajectory is chosen to compare with the initial position, and if it is better, it replaces the initial position. The migration loop continues until the algorithm satisfies the specified termination condition. SOMA has two different variations, the AllToOne (SOMA ATO) and AllToAll (SOMA ATA) techniques. SOMA ATO is when all individuals move towards the leader, whereas SOMA ATA is when all individuals move towards each other, regardless of whether the individual is better or worse.

2.2. iSOMA improvements

The iSOMA algorithm [12] is an improved version of the Self-Organizing Migrating Algorithm (SOMA). It addresses some of the limitations of SOMA by introducing new methods for the initialization, self-organizing, and migrating processes (can be seen in Fig. 1). The initialization process in iSOMA starts by generating a random population of potential solutions using uniformly distributed random numbers to scatter individuals in the whole given search space. This population is then evaluated by the fitness function, and the global best optimal solution is recorded.

The iSOMA's self-organizing process is different from the traditional SOMA's process. Instead of all individuals moving towards the leader, the iSOMA selects the best individuals in a group to move towards the best individual in another group. The iSOMA algorithm first randomly selects m individuals from the current population to form a subpopulation. Out of these m individuals, the best ($n \leq m$) are selected to become migrants. For each migrant, the algorithm randomly selects a second subpopulation of k individuals from the current population, which may include individuals already selected in the first subpopulation. The individual with the highest fitness value from the second subpopulation is chosen as the leader for that specific migrant. In the event that the chosen migrant is the same as the leader, the second-best individual from the second subpopulation is selected as the leader. This

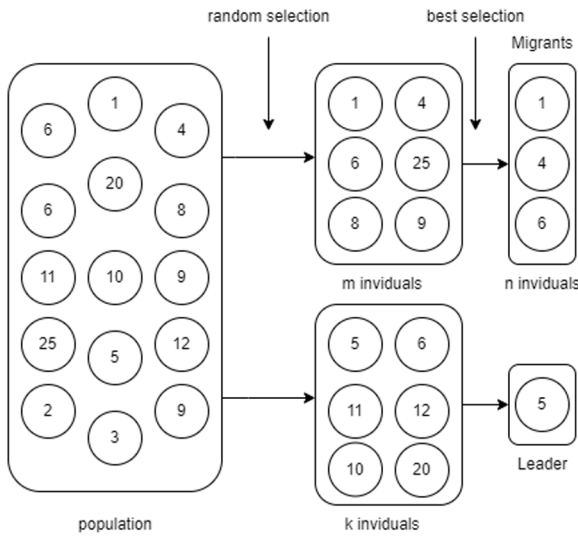


Fig. 2. The self-organizing process from [12].

helps in eliminating bad individuals and maintaining the diversity of the population. This can be seen in Fig. 1.

In the improved iSOMA algorithm, the migration process is modified so that if a migrant finds a better position, the algorithm updates the population with this new position immediately for that specific migrant. However, the entire iteration process continues for other migrants. This means that while the migrant that found a better position stops its jumping progress and updates the population, other migrants in the same iteration still complete their jumps as planned. This selective stopping and updating process allows for a more efficient exploration of the solution space, as it quickly incorporates promising positions found by individual migrants without halting the overall iterative process (see Fig. 2).

The replacement process in the iSOMA algorithm is the process of updating the population with better solutions. After each individual completes its migration, the best position in the moving trajectory is chosen to compare with the initial position. If the new position is better, it replaces the initial position, otherwise, the algorithm will skip the new position and continue the process for the remaining individuals. This ensures that the population is constantly updated with better solutions and helps in avoiding premature convergence. The replacement process is an important part of the iSOMA algorithm as it helps in finding the optimal solution to the given problem by gradually improving the population over multiple migration loops.

2.2.1. Parameters of iSOMA

There are several parameters that can be optimized in the iSOMA algorithm to improve its performance. These parameters include:

1. Migration step size (s): This parameter controls the granularity of the migration process and determines the size of the step that the migrants take towards the leader. Optimizing this parameter can help in finding the optimal balance between exploration and exploitation in the search space.
2. Perturbation parameter (PRT): This parameter is used to generate random perturbation moves and helps in maintaining diversity in the population. Optimizing this parameter can help in avoiding getting stuck in local optima and improve the chances of finding the global optimum. Simple adaptive PRT is introduced in iSOMA, which starts with a number close to 0 and ends with a number close to 1 (based on remaining FEs).

Table 1

Parameters of iSOMA algorithm and their values from original article.

Parameter	Explanation	Value
l	Path length	3
s	Step size	0.3
p	Population Size	100
m	Possible migrants	10
n	Migrants	5
k	Leaders	15
R	Part of population replaced	0.1
PRT	PRT parameter	$0.05 + 0.9(FEs / FEs_{max})$

3. Subpopulation sizes (m, n, k): These parameters control the number of individuals that are selected as migrants, leaders, and subpopulations in each migration loop. For problems containing many local traps, the values of m, n , and k should be small.
4. Path length (l): It refers to the total number of steps or distance that the migrants take towards the leader. A longer path length can help in exploring a larger portion of the search space, thus increasing the chances of finding the global optimum. However, a longer path length can also lead to increased computational time and may not be practical for large or complex problems. On the other hand, a shorter path length can help in reducing computational time but may not be able to explore the entire search space and may lead to getting stuck in local optima.

These parameters can be overviewed in Table 1 play a crucial role in determining the performance of the iSOMA algorithm and their optimal values can change depending on the problem at hand. By optimizing these parameters, we can improve the performance of the iSOMA algorithm and increase the chances of finding the global optimum. The whole algorithm can be seen in Algorithm 1.

Algorithm 1: iSOMA

```

1 Create and evaluate the initial population P;
2 Store the best individual as the global best one;
3 while stop condition not reached do
4   Select random m individuals from P;
5   Pick the best n out of m individuals as migrants;
6   for i = 1 to n migrants do
7     Select random k individuals from P;
8     Picked out the best of k as the leader;
9     if the leader is the migrant then
10      Change the leader to the second-best one in k;
11   while (njump ≤ Njump) and (no better position) do
12     Update PRT values;
13     The migrant moves to the leader;
14     Checking boundary;
15     Re-evaluate fitness function;
16     Updated this migrant;
17     Updated the global best position;
18   if the global best is not updated after t FEs then
19     Randomly replace x% of the population P;
20 return
```

2.3. Comparison with other algorithms

In addition, several algorithms that were competitive with iSOMA in the original article [12] employed adaptation based on previous success. One such algorithm is Success-history based parameter adaptation for differential evolution (SHADE) [29], which utilizes a distinct parameter adaptation mechanism based on a history, while maintaining a diverse set of parameters to guide control parameter adaptation as the

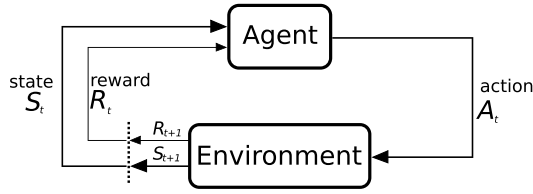


Fig. 3. Reinforcement learning.

search progresses for more robust search. Notably, SHADE was the only algorithm that relatively outperformed iSOMA in the benchmark.

Also, the algorithm DE with success-based parameter adaptation (DEsPA) [30] for CEC2015 learning-based optimization utilizes a success-based parameter adaptation with resizing population space. Another algorithm, the Tuning maturity model of ecogeography-based optimization on CEC 2015 single-objective optimization test problems (TEBO) [31], adapts its parameters for each test function. Similarly, the Self-adaptive Dynamic Particle Swarm Optimizer (SaDPSO) [32] also adapts its parameters for successful optimization.

There are numerous other algorithms that are used for CEC competitions and adapt their parameters based on previous performance for example [33–35]. To a certain extent, this can also be said about the iSOMA algorithm as it selects the best performing individuals. However, the algorithm does not utilize learning from history and most of its parameters are fixed.

2.4. Limitations of iSOMA

The iSOMA is a powerful optimization algorithm that has shown promise in various applications. However, like many optimization algorithms, it has its limitations, especially when dealing with ill-defined functions and high-dimensional spaces. Such functions can lead the algorithm to get stuck in local optima or fail to converge to a solution altogether. On the other hand, when dealing with high-dimensional spaces, the curse of dimensionality becomes a significant challenge. As the dimensionality of the problem increases, the search space grows significantly, making it difficult for the algorithm to explore the space efficiently. This can result in longer convergence times and an increased likelihood of missing the global optimum. Additionally, in high-dimensional spaces, the algorithm might require more iterations and a larger population size to achieve satisfactory results, which can be computationally expensive.

3. Reinforcement learning and swarm based algorithms

In this section will discuss the application of reinforcement learning, proximal policy optimization, and hybridization of swarm-based function optimizations.

3.1. Reinforcement learning

Reinforcement learning is a type of machine learning that involves training agents to make decisions in an environment by learning from the consequences of their actions [18] illustrated in Fig. 3. The agent receives feedback in the form of rewards or punishments, which it uses to adjust its behavior and improve its performance over time. Reinforcement learning is particularly useful for problems that involve sequential decision making, such as robotics, game playing [36], and control systems [37–39].

There are various approaches to reinforcement learning, including value-based methods, policy-based methods, and actor-critic methods [40]. Value-based methods, such as Q-learning [41], estimate the value of different actions and select the action with the highest estimated value. Policy-based methods, such as REINFORCE [42], directly

optimize the policy, or the mapping from states to actions. Actor-critic methods, such as A3C [36], combine both value-based and policy-based methods by having an actor network that selects actions and a critic network that evaluates the actions.

Overall, reinforcement learning is a powerful approach to optimization that can handle complex and dynamic environments [43,44]. Its ability to learn from experience and adapt to changing conditions makes it well-suited for real-world problems. However, it can be challenging to design the appropriate reward functions and properly train the agent, which requires careful experimentation and testing. Also most reinforcement learning is very sample inefficient and needs many sample to train properly [45].

3.1.1. Proximal policy optimization

Proximal Policy Optimization (PPO) [19] is a type of reinforcement learning algorithm that is particularly well-suited for problems that involve continuous action spaces. PPO is a policy-based method that aims to optimize the policy by maximizing the likelihood of the actions taken, while also ensuring that the policy does not deviate too far from the previous policy.

The algorithm is based on the idea of the “proximal” update, which means that it only updates the policy in a small region around the previous policy. This helps to prevent the policy from moving too far from the previous policy, which can lead to instability and poor performance. The algorithm also uses a technique called “trust region optimization”, which limits the size of the update to ensure that the policy does not deviate too far from the previous policy. The objective function for PPO can be seen in Eq. (1) as follows

$$L^{CLIP} = \hat{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (1)$$

In the context of this objective function, several key parameters are defined. Firstly, θ is the policy parameter. Secondly, $\hat{E}^t$ denotes the empirical expectation over timesteps, while $\hat{r}_t$ is the ratio of the probabilities under the new and old policies, respectively. Additionally, $\hat{A}^t$ represents the estimated advantage at time t , and ϵ is a hyperparameter, which is typically set to 0.1 or 0.2. This objective function introduces a method for conducting a Trust Region update that aligns with Stochastic Gradient Descent. It streamlines the algorithm by eliminating the necessity for the KL penalty and adaptive adjustments.

PPO has been applied to a wide range of problems, including robotic control, game playing, and decision making in complex systems. For example, in [46], PPO was used to train a robotic arm to perform a complex grasping task, achieving state-of-the-art performance, or solving rubik’s cube with robotic hand [47]. In [48], PPO was used to train a game-playing agent that was able to beat professional players in the game of Dota 2. Also PPO was used for guided exploration or [49] or solving quantum control tasks with continuous control parameters in numerical experiments [50].

Overall, PPO is a powerful and effective algorithm for solving problems with continuous action spaces. Its ability to update the policy in a small region around the previous policy helps to ensure stability and improve performance. However, it can be computationally intensive, and it may require a large number of samples to train effectively [45].

Reinforcement learning (RL) algorithms with deterministic actors (policy) commonly apply noise to the action space for exploration [51]. These exploration methods are either undirected or require extra knowledge of the environment.

3.2. Hybridizing swarm-based function optimization algorithms with reinforcement learning

Hybridizing swarm-based function optimization algorithms with reinforcement learning can combine the strengths of both approaches to optimize a problem. Swarm-based algorithms, such as particle swarm optimization (PSO), SOMA and ant colony optimization (ACO), are known for their ability to explore the search space and converge

to a global optimum. Reinforcement learning, on the other hand, is known for its ability to learn from experience and adapt to changing conditions.

By hybridizing these two approaches, the swarm-based algorithm can be used to explore the search space, while the reinforcement learning algorithm can be used to learn from the experience and adapt the parameters of the swarm-based algorithm. This can lead to a more efficient and robust optimization process.

For example, in [26] variant of gray wolf optimization (GWO) [52] that uses reinforcement learning principles combined with neural networks to enhance the performance is proposed. In GWO, a single parameter is used to control the exploration/exploitation rate (discrete action), which influences the performance of the algorithm. Based on this EGWO [53] algorithm is proposed, which modifies position-updated equation and add non linearity. Last reinforcement learning algorithm based on is RLGWO [24] is proposed to solve the UAVs three-dimensional path planning problem. RLGWO optimize each individual in isolation.

A Reinforcement Learning-based Genetic Algorithm (RL-GA), as proposed by Song et al. [54], addresses the Electromagnetic Detection Satellite Scheduling Problem by utilizing 15 distinct discrete actions derived from the preceding state. Complementarily, the Reinforcement Learning Sand Cat Swarm Optimization (RLSCSO) algorithm integrates Q-learning with a dual-action space encompassing exploitation and exploration, alongside a straightforward reward function. Yin et al. introduced RLAM [55], which leverages Deep Deterministic Policy Gradient (DDPG) and a continuous action space to enhance the Particle Swarm Optimization (PSO) algorithm. Diverging in approach, the HRLPSO algorithm [56] optimizes PSO through a reinforcement learning framework grounded in psychological principles. Furthermore, the Bee Swarm Optimization is augmented in the RSO algorithm [57], which employs the Q-learning algorithm.

Another set of hybridization of reinforcement learning and swarm-based function optimization algorithms are particle swarms. In article [23] a reinforcement learning-based communication topology in particle swarm optimization is proposed. Another one is a reinforcement learning level-based particle swarm optimization algorithm (RLLPSO) [58], where reinforcement learning is used for or level number control is employed to help improve the search efficiency of RLLPSO. SRL-PSO [59] is about swarm reinforcement learning algorithm based on PSO in which the personal best of each agent has a lifespan and is one of the older examples of this combination. RLEPSO [60] and RLPSO [61] are also examples of this approach, where RLPSO solves the problem of high-energy consumption in activated sludge wastewater treatment and RLEPSO uses reinforcement learning for pre-training in the design phase to automatically find a more effective combination of parameters. Authors of article [27] used complicated combination of REINFORCE algorithm with approximation of the actual objective value for noisy problems. Reinforcement Learning-based Memetic Particle Swarm Optimizer (RLMPSO) is proposed using memetic particles. Also in a NRLPSO reinforcement using Q-learning is used, where a reinforcement learning velocity vector balances exploration and exploitation.

For ant colony is a example ANT-Q [69] or Quantum ant colony algorithm (ACA) [64] for Q-learning of Quantum System. For Evolutionary Algorithms there is a RL controller (EA with RL) [65]. Different approach took Prioritized experience replay for parallel hybrid evolutionary and swarm algorithms PESA [70] that propose a combining prioritized replay of reinforcement learning with hybrid evolutionary algorithms for application to nuclear fuel.

In general, hybridizing swarm-based function optimization algorithms with reinforcement learning can provide a powerful and effective solution for solving optimization problems. It combines the ability of swarm-based algorithms to explore the search space with the ability of reinforcement learning to learn from experience and adapt to changing conditions. However, it can be challenging to design the appropriate reward functions and properly integrate the two algorithms, which requires careful experimentation and testing. Summary of algorithms mentioned in article can be seen in Table 2.

4. Proposed methods

In this section we will describe our proposed method to integrate PPO to control parameters of algorithm of iSOMA.

4.1. Optimization of the parameters of the algorithm

In this study, we propose a new algorithm called iSOMA-RL, which combines the iSOMA algorithm with the Proximal Policy Optimization (PPO) method for the optimization of algorithm's parameters in swarm intelligence. The iSOMA algorithm is a population-based optimization algorithm that mimics the competition-cooperation behavior of individuals to find the optimal solution. By using PPO, a state-of-the-art reinforcement learning technique, iSOMA-RL can dynamically and continuously optimize algorithm's parameters that are crucial to determining algorithm performance. Additionally, the PPO algorithm is always created anew for each problem (episode) and test, with no pretraining done in order to remove bias from pretraining. The advantage of this approach is that it requires no additional pretraining. The only fixed additional workload needed is during the optimization process, and it is independent of the function being optimized. Our code implementing environment and iSOMA-RL can be found on <https://github.com/Lukykl1/iSOMA-RL>.

4.2. Implementation of PPO and implementation details

In this study, we implement iSOMA-RL using Stable Baselines3 [71] and OpenAI's gym environment [72]. The Stable Baselines3 library contains various RL algorithms and also includes PPO. The OpenAI's gym environment interface is used to interface with the PPO algorithm, and it serves as a standardized way to interface with test environments like games, stock trading and others.

We use the PPO algorithm without any modifications and use default values as can be seen in Table 3 as is implemented in Stable Baselines3.

The PPO algorithm has several important parameters (as can be seen in Table 3) that affect its performance and efficiency. Hyperparameter tuning is usually recommended to achieve better results, but we did not employed that to do not introduce any bias. Stable Baselines3 provides a framework for this, named "rl-zoo".

The OpenAI's gym environment defines four methods:

1. step - for performing one step in the environment
2. reset - for resetting the environment to an initial state
3. close - for freeing resources (if not used)
4. render - for visualizing what the agent sees, in this case, it is ASCII text of the observation space.

Additionally, OpenAI's gym environment also defines the dimension and types of spaces and rewards.

In our defined environment, a single step consisted of 10 function evaluations, which is equivalent to ten steps of iSOMA with parameters set by the PPO algorithm. After each step, a reward is calculated, and the PPO network is updated. Then, another observation space is sampled (refer to Section 4.2.2) and an action is taken (which involves continuous values depending on the variant used).

4.2.1. PPO, Actor and Critic network

The Proximal Policy Optimization algorithm (overview on Fig. 4) combines ideas from A2C (which involves multiple workers) and TRPO (which utilizes a trust region to enhance the actor). In Stable Baselines 3, Proximal Policy Optimization (PPO) is implemented using an actor-critic architecture. This approach comprises three main components: a feature extractor, an actor network, and a critic network. The main idea is that after an update, the new policy should not be too far from the old policy. For this reason, PPO uses clipping to avoid excessively large updates.

Table 2
Summary of the swarm-based optimization algorithms with reinforcement learning.

Algorithm (RL type)	Actions	Reward type
QLPSO [62] Q-learning	Discrete	Discrete 4 variants: 2, 1, 0, -2
RLLPSO [58] Q-learning	Discrete	Continuous (difference between positions)
RLGWO [24] Q-learning	Discrete (4 actions)	Discrete (1 if fitness improved, else -1)
EGWO [53] Q-learning	Discrete (3 actions)	Discrete (1 if fitness improved, else -1)
RLPSO [61] Elite network model	Continuous (velocity prediction)	Discrete (1 fitness improved, else -1)
RLEPSO [60] DDPG [63]	Continuous	Discrete (1 fitness improved, else -1)
PSO with RL [27] REINFORCE	Continuous	Continuous
SRL-PSO [59] Q-learning	Discrete (4 actions)	Discrete (+100 goal is met, else -1)
RLMPSO [27] Q-learning	Discrete (5 actions)	Discrete (+1 fitness improved, else -1)
ACA [64] Q-learning	Continuous	Continuous increment of pheromone
EA with RL [65] SARSA [66]	Discretized space	Continuous normalized to effort
NRLPSO [67] Q-learning	Discrete (4 actions)	Discrete -2, -1, ..., 2 based on cost and diversity
RL-GA [54] Q-learning	Discrete (15 actions)	Continuous Difference between actual and previous fitness
RLSCSO [68] Q-learning	Discrete (2 actions)	Discrete (+1 fitness improved, else -1)
RLAM [55] DDPG	Continuous	Discrete (+1 fitness improved, else -1)
RSO [57] Recursive Q-learning	Unspecified	Continuous recursive reward

Table 3
Important parameters of PPO.

Parameter	Explanation	Default Value
policy	The policy model used	MultiInputPolicy
learning_rate	Learning rate	0.0003
n_steps	Steps per update	2048
batch_size	Minibatch size	64
n_epochs	Number of optimization epochs	10
gamma	Discount factor	0.99
gae_lambda	Bias-variance trade-off factor	0.95
clip_range	Clipping parameter	0.2
clip_range_vf	Clipping parameter for value function	None
normalize_advantage	Advantage normalization	True
ent_coef	Entropy coefficient	0.0
vf_coef	Value function coefficient	0.5
max_grad_norm	Max gradient norm	0.5
use_sde	Using gSDE instead of action noise	False
sde_sample_freq	Noise matrix sampling frequency	-1

For environments with 1D observation spaces, a two-layer fully connected neural network is employed, with each layer containing 64 units. The Critic network (Table 4) is responsible for estimating the value function, which can be either the action-value (Q value) or the state-value (V value). On the other hand, the Actor network (Table 4) updates the policy distribution based on the guidance provided by the Critic, typically involving the use of policy gradients.

In the case of PPO, when dealing with continuous action spaces, actions are clipped both during training and testing to prevent out-of-bounds errors. The activation function used is tanh. The optimizer for the network is ADAM.

Table 4
Example Critic network and Action network.

Layer	Critic network		Action network	
	Operation	Parameters	Operation	Parameters
1	Linear	in_features=100, out_features=64	Linear	in_features=100, out_features=64
2	Tanh()	-	Tanh()	-
3	Linear	in_features=64, out_features=64	Linear	in_features=64, out_features=64
4	Tanh()	-	Tanh()	-
5	Linear	in_features=64, out_features=1	Linear	in_features=64, out_features=2

4.2.2. Observation space

The observation space used in the algorithm includes the history of actions taken (HistoryLen (h_l) actions back), history of difference in fitness (h_l actions back), and history of fitness (h_l actions back). h_l is a variable that indicates how many steps in history should be available to the algorithm. The observation space can be seen in Table 5. The history is added to enable learning from previous actions, and our preliminary testing showed that including history improves the performance of the algorithm.

4.2.3. Action space

The iSOMA algorithm entails numerous parameters for optimization. However, attempting to optimize them all simultaneously using RL proved unsuccessful due to the vast search space and the constraints imposed by limited amount of function evaluations. In our approach,

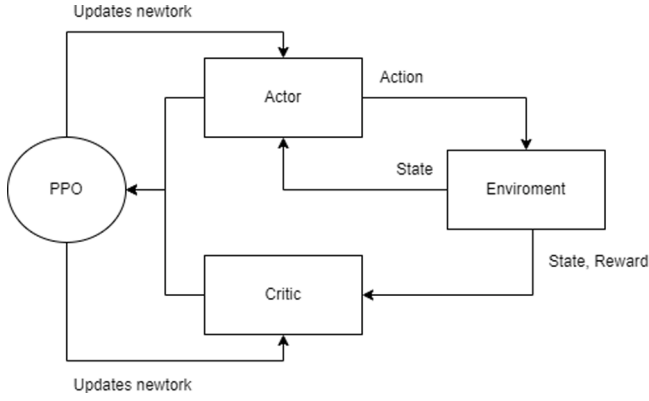


Fig. 4. Overview of PPO algorithm.

Table 5

Observation space.

Variable	Shape
Fitness	$(h_i, 1)$
Difference in fitness	$(h_i, 1)$
Action n	$(h_i, 1)$
Action $n + 1$	$(h_i, 1)$

we utilized continuous actions that were sampled at each step. We also constrained the action space within reasonable lower and upper bounds.

4.2.4. Training of PPO

We did not undergo any pretraining. For instance, while PPO is engaged in training, the iSOMA algorithm optimizes the function and uses function evaluations. This approach conserves computation and eliminates any bias from encountering similar functions. In this manner, PPO learns a singular function and strives to determine the optimal parameters using a reward function while seeking the best value of the function. The disadvantage is that PPO is constrained by the number of function evaluations (FEs), so it should perform better on more complex problems (with more dimensions).

Such as the PPO algorithm learns online and trains during the optimization process. This enables PPO to dynamically optimize the function based on previous attempts. However, a drawback is that it cannot learn from past functions, which eliminates any potential bias that might arise from pretraining. This can be seen in Fig. 5, that shows how parameters are being optimized.

4.2.5. Three different parameter sets to optimize using PPO

The iSOMA algorithm has many parameters to optimize, but because we were not successful in optimizing them all at once using RL, we divided them into three categories, which we benchmarked and evaluated. The actions X is a vector of actions, where x_y has a range of $[-1, 1]$. Overview of these three variants can be seen in Fig. 6.

The modification of these parameters is governed by the Proximal Policy Optimization RL algorithm, which dynamically adjusts them, allowing for either a continuous increase or decrease. Each action in the vector X is subsequently employed in the computation of iSOMA parameters, as detailed in Tables 6, 7, and 8.

As an illustrative example, the calculation for a particular iSOMA parameter might be represented as $0.35 + 0.25x_0 + 0.2 \left(\frac{FEs}{FEs_{max}} \right)$, as shown in Table 6. Here, FEs denotes the number of function evaluations, and FEs_{max} is the maximum allowed evaluations. The term $0.25x_0$ in the equation signifies the influence of the action x_0 on this parameter. The magnitude and direction of change in x_0 – whether it increases or decreases – are entirely contingent upon the PPO algorithm's learned behavior or its experimentation with new strategies. Thus, the PPO

Table 6

Special parameters of iSOMA-RL algorithm for PRT variant.

Parameter	Explanation	Value
PRT	PRT parameter	$0.35 + 0.25x_0 + 0.2(FEs/FEs_{max})$
h_i	History length	25

Table 7

Special parameters of iSOMA-RL algorithm for subpopulation variant.

Parameter	Explanation	Value
p	Population Size (unchanged)	100
m	Possible migrants	$1 + P(0.31 + 0.25x_2)(0.31 + 0.25x_1)$
n	Migrants	$1 + P(0.31 + 0.25x_2)$
k	Leaders	$1 + P(0.31 + 0.25x_0)$
h_i	History length	25

algorithm plays a crucial role in dynamically tuning the action x_0 to enhance the overall efficacy of the iSOMA algorithm.

The parameters of the iSOMA algorithm are divided into three categories and optimized by the PPO algorithm:

1. PRT Optimizing — the PPO algorithm optimizes the PRT parameter of the iSOMA
2. Subpopulation Optimizing — the PPO algorithm optimizes the m , n , k parameters of the iSOMA
3. Path Length and Step Optimizing — the PPO algorithm optimizes the l and s parameters of the iSOMA

4.2.6. PRT optimizing

This variant optimizes the PRT parameter, which should influence the exploitation or exploration of the iSOMA algorithm [12]. Additionally, the PRT parameter was already naively adaptive in the original iSOMA algorithm. In this variant, adaptation based on the remaining FEs was slightly kept by keeping PRT . The parameters of this variant are shown in Table 6. On Fig. 7 can be seen a single run of iSOMA-RL optimizing a Schwefel's Function.

4.2.7. Subpopulation optimizing

This variant optimizes the m , n , and k parameters. As stated in the original article [12], the values of m , n , and k affect the iSOMA algorithm's ability to explore and exploit the search space. When solving complex problems, a larger number of moves between random individuals is performed to improve the algorithm's exploring ability in the search space. However, for simpler problems, larger values of m , n , and k are used to direct the algorithm's focus towards better individuals, thus increasing its exploiting ability in the promising searched space. These parameters play a significant role in determining the algorithm's performance. The parameters of this variant can be seen in Table 7. On Fig. 8 can be seen a single run of iSOMA-RL optimizing a Schwefel's Function. In Fig. 8, the employed reward function is Complex Reward 2. As evidenced by the figure, there is a positive correlation between the increase in the reward and the decrease in the cost function. Furthermore, this function demonstrates a scaling relationship with the count of Function Evaluations (FEs). Notably, in the later stages of the search process, the reward escalates, thereby incentivizing the exploration of additional actions to discover new minima.

4.2.8. Path length and step size optimizing

The step and path length in iSOMA are also important parameters that affect the algorithm's performance and efficiency.

The step size determines the distance between each position on an individual's jumping path. A larger step size allows for a greater exploration of the search space but also increases the risk of missing important solutions. A smaller step size increases the chances of finding a better solution, but also increases the number of function evaluations required.

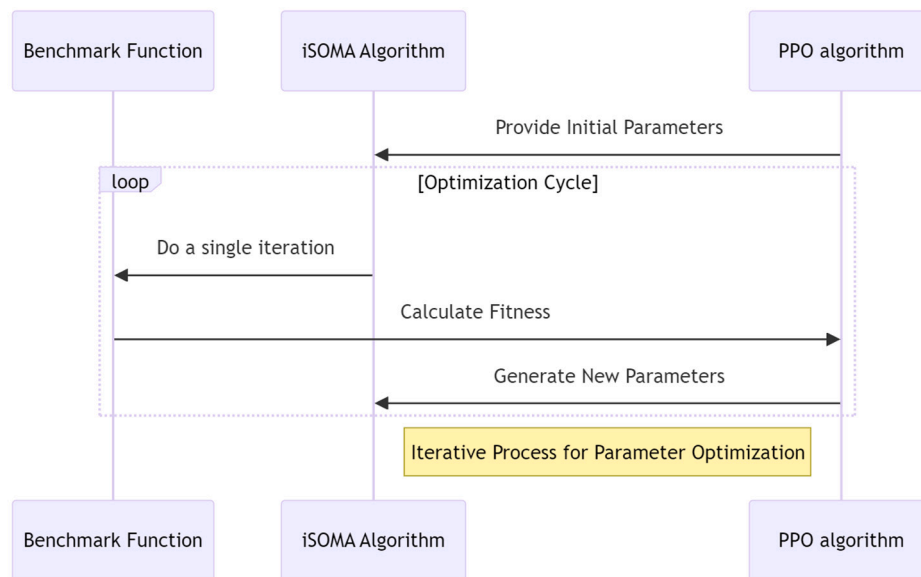


Fig. 5. Simple schematic diagram of single function parameter optimization using iSOMA-RL.

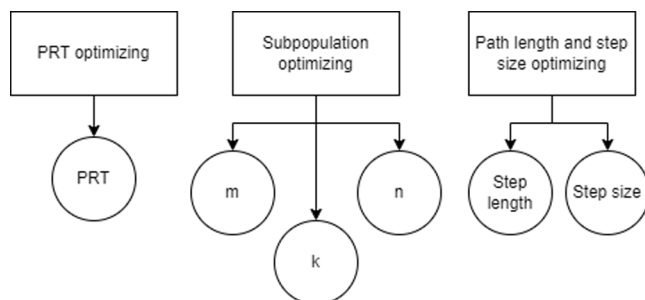


Fig. 6. Different parameter sets.

The path length determines the number of positions on an individual’s jumping path. A larger path length allows for a greater exploration of the search space but also increases the number of function evaluations required. A smaller path length reduces the number of function evaluations, but also limits the exploration of the search space.

In general, optimizing step and path length in iSOMA is a trade-off between exploration and exploitation. A balance should be found that allows the algorithm to efficiently explore the search space while also finding good solutions. The parameters of this variant can be seen in Table 8. On Fig. 9 can be seen a single run of iSOMA-RL optimizing a Schwefel’s Function. In Fig. 9, the same Complex Reward 2 function is utilized; however, the distribution of rewards appears more scattered and sparse. This variation is attributed to the influence of the ‘move’ and ‘jump’ parameters on the function’s performance. These parameters significantly affect the identification of viable solutions. As a result, the figure illustrates a pattern where better solutions are intermittently identified amidst a backdrop of less valid options. The sporadic nature of high rewards in this scenario underscores the dynamic interaction between these parameters and the search algorithm’s effectiveness in navigating the solution space.

4.3. Reward function

We also explored various reward functions, recognizing their potential significance. It is worth noting that there is no consensus in the literature regarding the optimal reward function for function optimization.

Table 8
Special parameters of iSOMA-RL algorithm for Path length and step size variant.

Parameter	Explanation	Value
l	Path length	$(10 + 3x_1)(0.3 + 0.025x_0)$
s	Step size	$0.3 + 0.025x_0$
h_l	History length	25

Benchmarking different reward functions is beneficial because it allows us to evaluate how the choice of reward function affects the performance of the algorithm. The reward function is an essential component of reinforcement learning, as it guides the agent to take actions that maximize the cumulative reward. A good reward function should be able to effectively guide the agent towards the optimal solution while avoiding suboptimal solutions.

By benchmarking different reward functions, we can compare their performance and identify which reward function leads to the best results. This information can be used to improve the performance of the algorithm and can also help in understanding the behavior of the algorithm in different scenarios. Additionally, by benchmarking different reward functions, we can also identify which reward functions are sensitive to certain problem characteristics, such as the dimensionality of the problem, the type of function, and the presence of local optima. This information can be used to design more effective reward functions for specific types of problems.

In this framework, various parameters are defined with specific roles. The parameter r represents the reward value, and d stands for the difference between the current best solution and the previous best solution. The total difference is denoted by t , while f signifies the current number of function evaluations. Additionally, b refers to the number of function evaluations before the current evaluation, and FEs_{max} represents the maximum number of function evaluations allowed in the process. These parameters collectively contribute to the functioning and performance of the algorithm in question.

We define these 5 reward functions: Simple, Difference, No penalty, two versions of Complex reward.

4.3.1. Simple reward function

One common approach for defining a reward function in RL enhanced optimization algorithms is to assign a reward of 1 if the fitness improves and -1 if it does not. This approach has been used in various

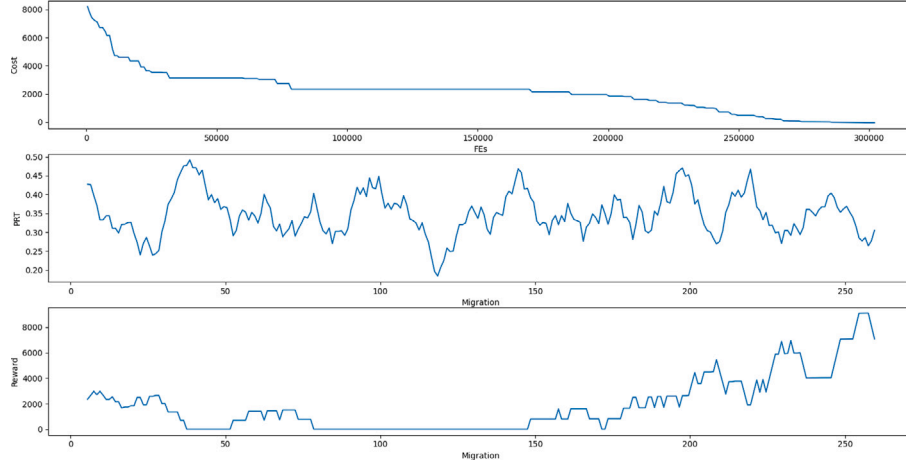


Fig. 7. Example of iSOMA-RL optimizing PRT.

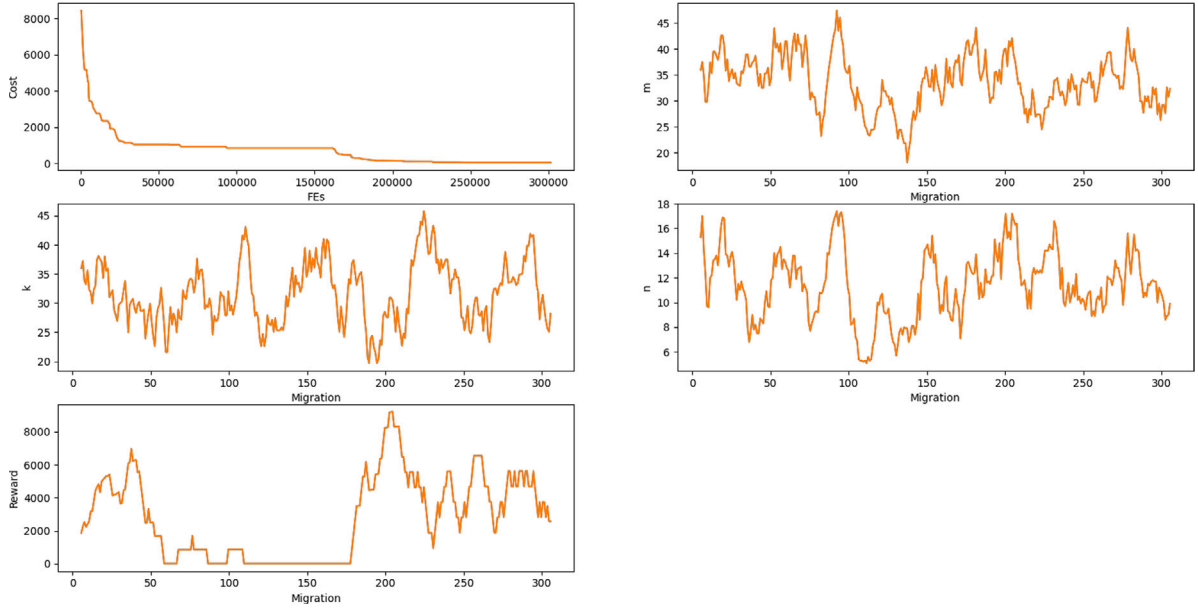


Fig. 8. Example of iSOMA-RL optimizing subpopulation and its parameters.

algorithms such as RLWGO, EGWO, RLEPSO, or RLMPSO. However, this approach can lead to a problem in the early iterations where the PPO algorithm prioritizes small but steady improvements in fitness over more significant but less consistent ones.

4.3.2. Difference reward function

Another approach for defining the reward function is to use the difference between the new and previous best fitness as the reward. This approach is defined in Eq. (2) and was used in the RLLPSO algorithm. Similar rewards have also been used in ACA and PSO with RL.

$$r = d \quad (2)$$

4.3.3. Reward function without penalty

Additional approach for defining the reward function is to use the difference between the new and previous best fitness, multiplied by the total difference, as the reward. This approach assigns a higher value to solutions that have a higher difference from the previous best solution. This approach is defined in Eq. (3). The multiplication by the total

difference should give a greater reward for smaller differences later in the test run, as the algorithm is looking for the best fitness in smaller steps.

$$r = d \cdot t \quad (3)$$

4.3.4. Complex reward function

Next approach for defining the reward function is to use the difference between the new and previous best fitness, multiplied by the total difference, as the reward. This approach assigns a higher value to solutions that have a higher difference from the previous best solution. The final reward value is then subtracted by the ratio of the used function evaluations over the maximum number of function evaluations. Additionally, if the value of the reward is greater than the total difference, it is set to the total difference. This approach is defined in Eq. (4). This reward function gives a penalty when more function evaluations (FEs) are used. This approach was used in the EA with RL algorithm.

$$r = d \cdot t - \frac{f - b}{FEs_{max}} \quad (4)$$

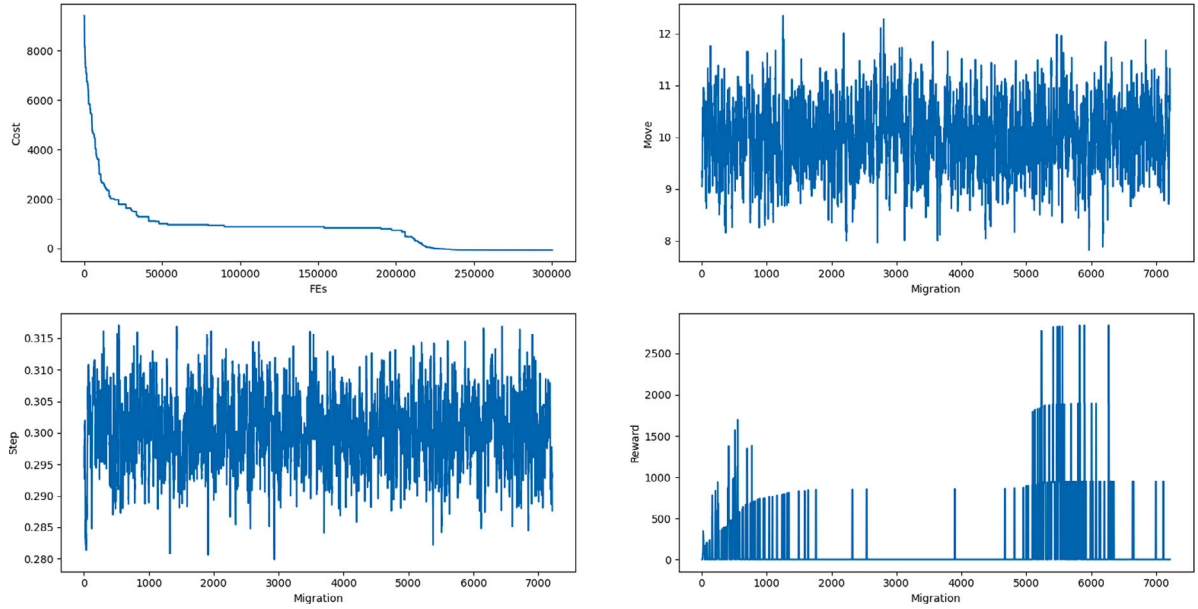


Fig. 9. Example of iSOMA-RL optimizing path length and step size optimizing.

4.3.5. Complex reward function 2

Another variation of the reward function is to use the difference between the new and previous best fitness, multiplied by the total difference, as the reward. The maximum number of function evaluations in previous equation is squared and also the total difference is squared. This change prioritizes even more smaller differences later in the run. Additionally, this change decreases the penalty for using more function evaluations (FEs) in comparison to the previous equation. This approach is defined in Eq. (5).

$$r = d \cdot t^2 - \frac{f - b}{FE_{max}^2} \quad (5)$$

4.4. Benchmark for best performing variant and reward function

In order to effectively evaluate the performance of the iSOMA algorithm, we utilized three commonly used test suites from the IEEE Congress on Evolutionary Computation (IEEE CEC). These test suites include a total of 72 functions, which are divided into the following benchmark sets:

1. The IEEE CEC 2013 Special Session on Real Parameter Single Objective Optimization, which consists of 28 functions. The details of this benchmark set can be found in [73].
2. The IEEE CEC 2015 Special Session on Bound Constrained Single objective Computationally Expensive Numerical Optimization, which consists of 15 functions. The details of this benchmark set can be found in [74].
3. The IEEE CEC 2017 Special Session on Bound Constrained Single objective Computationally Expensive Numerical Optimization, which consists of 29 functions. The details of this benchmark set can be found in [75].

These single objective benchmark problems were chosen for evaluation because they serve as the foundation for research on more complex optimization problems, such as multi-objective, dynamic, niching composition, and computationally expensive problems. They are classified into various types of functions including unimodal, basic multimodal, simple multimodal, hybrid, and composition, (non-)separable, shifted, and rotated functions that are challenging enough to evaluate the performance of an optimization algorithm. These functions are widely used in the research community and provide a standardized way to evaluate and compare the performance of different optimization algorithms.

Table 9

Summary of benchmark against original iSOMA.

iSOMA variant	Dimension	FEs
Original iSOMA	D	$D \cdot 10^4$
PRT Optimizing	D	$D \cdot 10^4$
Path length and Step Optimizing	D	$D \cdot 10^4$
Subpopulation Optimizing	D	$D \cdot 10^4$

Table 10

RL reward function benchmark.

Reward function	Dimension	FEs
Simple reward	10	$10 \cdot 10^4$
Difference	10	$10 \cdot 10^4$
No penalty	10	$10 \cdot 10^4$
Complex reward	10	$10 \cdot 10^4$
Complex Reward 2	10	$10 \cdot 10^4$

4.4.1. Used library for test functions

For the implementation of these test functions, we utilize the Opfunu library [76], which has a clean interface and implements many CEC test suites. We use the original bounds, which are specific for each function and in all cases of test suites, it is a range of $[-100, 100]$. Also, we benchmark each test function at least 51 times to ensure the robustness of the results.

4.4.2. Benchmark settings

We benchmark the original iSOMA algorithm against our three variants with these single objective benchmark problems. For this, we choose a 30 dimensions and set the number of function evaluations (FEs) to $30 \cdot 10^4$. The parameters of the original iSOMA are set to the same as in the article [12]. Also, see Table 1 for parameter definitions. The summary of the benchmark can be seen in Table 9. Additionally, we also benchmark in the case of 10 dimensions, where we omit the CEC 2015 test suite. For these tests, we used the Complex Reward 2 function.

For the evaluation of different reward functions, we use a 10 dimensions and set the number of function evaluations (FEs) to $10 \cdot 10^4$. We limit ourselves to only the CEC 2013 test suite. We benchmark all variants together. The summary of the benchmark can be seen in Table 10.

To assess whether the differences in results between the different variants are statistically significant, the Wilcoxon rank-sum test (WRT) was applied at a significance level of 5% [77]. This statistical test is used to determine whether two sets of observations come from the same distribution and to compare their medians. The results of the WRT can be used to determine if any observed differences in performance between the variants are statistically significant or if they can be attributed to random variations.

4.5. Benchmark for efficiency

To assess the efficiency of our proposed improvement to iSOMA against other algorithms, we selected a set of famous and we could say state-of-the-art algorithms and conducted benchmarking in a similar fashion to the variants. We performed these benchmarks in both 10 and 30 dimensions, applying identical constraints (such as same max function evaluations count), using the 2013, 2015, and 2017 CEC benchmark suites. Additionally, we employed the Wilcoxon rank-sum test (WRT) at a significance level of 5%.

We benchmarked iSOMA-RL with subpopulation optimization, as it demonstrated the highest performance among the variants. We did not expect it to outperform all of them, as our optimization efforts focused solely on refining the parameters of iSOMA, without altering the fundamental workings of the algorithms. For this comparison, we chose from the differential evolution family (notably the most performant, surpassing iSOMA in some benchmarks) including the older L-SHADE [78], as well as the more recent iL-SHADE [79] and MPEDE [80]. Additionally, we considered other algorithms CMA-ES [81], ABC [82], and CL PSO [83]. These functions were sourced from the MEALPY library [84], except for the more recent iL-SHADE and MPEDE, which required the PyADE library.¹

1. **L-SHADE [78] (Large Population Success-History Adaptation Differential Evolution)**: It is a variant of the differential evolution algorithm that focuses on large population sizes.
2. **iL-SHADE [79] (Improved Large Population Success-History Adaptation Differential Evolution)**: An iL-SHADE algorithm is an extended version of the L-SHADE algorithm.
3. **MPEDE [80] (Differential evolution algorithm with multi-population cooperation and multi-strategy integration)**: It is a multi-population based DE variant that realize a dynamic ensemble of multiple mutation strategies.
4. **CMA-ES [81] (Covariance Matrix Adaptation Evolution Strategy)**: This algorithm is an evolutionary algorithm for numerical optimization that uses a probabilistic model to adapt the covariance matrix of the search distribution.
5. **ABC [82] (Artificial Bee Colony)**: Inspired by the foraging behavior of honeybees, this algorithm simulates the search for the best solution using a population of artificial bees.
6. **CL PSO [83] (Comprehensive Learning Particle Swarm Optimization)**: An extension of the classic Particle Swarm Optimization (PSO) algorithm, incorporating comprehensive learning strategies for enhanced performance.
7. **GA (Genetic Algorithm)**: A heuristic optimization algorithm inspired by the process of natural selection. It evolves a population of candidate solutions over multiple generations to find the best solution.

Table 11

Comparison of wins, losses, and draws of iSOMA-RL for different variants against original iSOMA with statistical significance (WRT 5%), 30 dimensions.

Algorithm	Better	Worse	Draws	Total
Subpopulation optimizing	39	4	29	72
Path length and Step Optimizing	38	5	29	72
PRT optimizing	39	5	28	72

4.6. Benchmarking overhead of using reinforcement learning

To gain a better understanding of the practicality and negative impacts of integrating reinforcement learning into the iSOMA, we conducted a benchmarking analysis. To demonstrate the potential drawbacks of utilizing reinforcement learning, we conducted a benchmarking analysis on a desktop personal computer equipped with an AMD Ryzen 5 3600 6-Core processor and 32 GB of RAM.

We executed each benchmark function ten times, both with and without reinforcement learning, and recorded the results using the timeit library. The purpose of this experiment was to evaluate the practicality and potential negative impact of integrating reinforcement learning into the iSOMA. We also aimed to determine the practicality of implementing this approach and how its effectiveness is influenced by the complexity of functions, number of FEs, and dimensions of the functions to optimize.

In this way, we can determine whether the additional overhead is constant or dynamic. Additionally, we can assess the overload incurred by using PPO in conjunction with iSOMA. This will demonstrate the applicability of employing the proposed method and help validate our theory that PPO introduces only a constant overhead. Such insights would be particularly beneficial for more complex functions, where function evaluation is more costly.

5. Results and discussion

In this section, we present the results and discussion of our study, where we proposed several variants of the iSOMA algorithm and compared their performance to the original iSOMA algorithm. To evaluate the effectiveness of the proposed variants, we conducted experiments and compared the results in terms of their ability to optimize a given function. We analyzed various metrics such as convergence speed and solution quality to assess the effectiveness of the proposed methods. Moreover, we compared the rewards function used in the proposed variants to evaluate their impact on the algorithm's performance. We also analyzed the overhead of using reinforcement learning in the proposed variants. We presented the results for different dimensions of the optimization problem.

5.1. Comparison of proposed method to original iSOMA

In this study, several variants of the iSOMA algorithm were proposed and compared to the original iSOMA algorithm. The performance of the proposed variants was evaluated by comparing the results to the original iSOMA algorithm using a set of benchmark functions in both 30 and 10 dimensions. All results in terms of mean and standard deviation can be found in supplemental file.

5.1.1. Comparison for 30 dimensions case

Based on the results shown in Table 11, it can be seen that the subpopulation optimizing variant performed the best, with 38 wins, 5 losses and 4 draws, out of a total of 72 test functions. The path length and step optimizing variant had 37 wins, 6 losses and 4 draws, while the PRT optimizing variant had 38 wins, 6 losses and 4 draws.

It is clear that the subpopulation optimizing variant had the most wins, and the least losses, which indicates that it is the most effective variant among the three variants. The path length and step optimizing

¹ <https://github.com/xKuZz/pyade>

Table 12

Comparison of wins, losses, and draws for different variants of iSOMA-RL against original iSOMA using simple mean, 30 dimensions.

Algorithm	Better	Worse	Draws	Total
Subpopulation optimizing	48	20	4	72
Path length and Step Optimizing	54	14	4	72
PRT optimizing	49	19	4	72

Table 13

Examples of significant improvements or degradation against original iSOMA, 30 dimensions (S. and R. means switched and rotated).

Function	Original	RL used	Change
Rastrigin's F.	2,48E+01	8,23E+02	33%
Schwefel's F.	1,90E+03	3,63E+01	2%
Composition F. 2	1,81E+03	1,11E+02	6%
Composition F. 9	2,05E+09	2,77E+03	0%
Composition F. 10	7,57E+09	9,72E+03	0%
S. and R. Rosenbrock's Function	6,08E+03	3,73E+01	1%
S. and R. Lunacek Bi-Rastrigin's F.	3,21E+04	4,92E+00	0%
S. and R. Non-Continuous Rastrigin's F.	1,81E+00	0,00E+00	Found best
S. and R. Levy F.	2,40E+01	0,00E+00	Found best
S. and R. Schwefel's F.	6,28E+03	2,13E+03	34%
Hybrid F. 4	4,38E+03	5,80E+03	132%
Hybrid F. 8	1,18E+03	1,48E+03	125%

variant had slightly fewer wins and more losses, while the PRT optimizing variant had the same number of wins as the subpopulation optimizing variant, but with more losses.

Based on the results shown in Table 12, it can be seen that the path length and the step optimization variant performed the best, with 54 wins, 14 losses, and 4 draws, of a total of 72 test functions. The subpopulation optimization variant had 48 wins, 20 losses, and 4 draws, while the PRT optimization variant had 49 wins, 19 losses, and 4 draws. Also in Table 13 are shown examples of very significant improvements over original iSOMA. Only in a case of Hybrid Function 4 was iSOMA-RL about 33% worse than original and in the case of Hybrid Function 8 it was 25% degradation. Other degradations are lower than the 15% degradation.

It is clear that the path length and step optimization variant had the most wins and the least losses, indicating that it is the most effective variant among the three variants. The subpopulation optimization variant had fewer wins and more losses compared to the path length and step optimization variant, and the PRT optimization variant had similar numbers of wins but with more losses.

It is important to note that this table does not reflect statistical significance, it only compares the mean of wins, losses, and draws. The previous Table 11 is more important because it has statistical significance using the Wilcoxon rank sum test.

Overall, in the 30-dimension case, the proposed variants showed an improvement in the optimization performance compared to the original iSOMA algorithm. Specifically, the subpopulation optimization variant had the most wins and the least losses, the path length and step optimization variant had the smallest wins and the most losses, while the PRT optimization variant had a similar number of wins, but with more losses.

5.1.2. Comparison for 10 dimensions case

In contrast, the results for the case of 10 dimensions showed a more mixed performance, with the subpopulation optimization variant having the most wins and the least losses, the Path length and Step Optimizing variant having fewer wins and more losses, and the PRT optimization variant having the least wins and the most losses as it probably did not learn properly to optimize the parameter *PRT*. It is worth noting that in the 10-dimension case, the difference between the variants is not as large as in the 30-dimension case, and further experiments and analysis may be necessary to determine the best variant for this specific case. See Table 14.

Table 14

Comparison of wins, losses, and draws for different variants against original iSOMA with statistical significance (WRT 5%), 10 dimensions.

Algorithm	Better	Worse	Draws	Total
Subpopulation optimizing	24	5	29	58
Path length and Step Optimizing	18	10	30	58
PRT optimizing	15	21	22	58

Table 15

Algorithm performance comparison in 30 dimensions (Win means that iSOMA-RL performed better).

Algorithm	iSOMA-RL		iSOMA		Improvement	
	Win	Loss	Win	Loss	Win	Loss
ABC	70	0	62	5	+7	-5
CL PSO	59	9	51	18	+8	-9
CMA-ES	46	22	37	30	+9	-8
L-Shade	14	44	5	53	+9	-9
iL-Shade	26	36	15	48	+11	-12
MPEDe	34	29	18	43	+16	-14
GA	62	9	54	17	+8	-8

The reason for the better performance of the proposed variants in the 30 dimensions case compared to the 10 dimensions case could be attributed to the increased complexity of the problem. In general, reinforcement learning algorithms such as PPO tend to perform better on more complex problems as they can better handle the increase in the number of variables and interactions between them. Additionally, it is possible that the number of function evaluations in the case of 10 dimensions was too small to fully optimize the performance of the variants, resulting in less significant differences between the variants. It is also possible that the optimization problem with 10 dimensions is less complex and it is easier to find the optimal solution with simpler optimization methods. The RL optimization of subpopulations is still the best option of our variants.

5.2. Comparison of results against other types of algorithm

The table presented in Table 15 provides a comparative analysis of the performance of various optimization algorithms in 30 dimensions, namely iSOMA-RL, iSOMA, ABC, CL PSO, CMA-ES, L-Shade, iL-SHADE, MPEDe, and GA. The metrics evaluated include the number of victories (Win) and losses (Loss) for each algorithm.

Notably, it is evident that iSOMA-RL outperforms iSOMA across all evaluated scenarios in an average of 12 test cases. This is particularly noteworthy given that iSOMA is recognized for its robustness in challenging optimization tasks, as demonstrated by its performance against the formidable L-Shade algorithm. In this context, iSOMA-RL exhibits a notable improvement over iSOMA, signifying its enhanced capability in tackling complex optimization problems.

Furthermore, the results reveal that the ABC algorithm experienced a complete defeat when compared to iSOMA-RL, indicating the latter's superior effectiveness in optimizing the given set of functions. It is also worth mentioning that other algorithms, such as CL PSO, CMA-ES, and GA, demonstrated varying degrees of performance, with iSOMA-RL consistently outperforming iSOMA.

In the 10-dimensional context, as depicted in Table 16, the performance of various optimization algorithms, including iSOMA-RL and iSOMA, was evaluated. Comparatively, it is evident that iSOMA-RL still exhibits superior performance over iSOMA in this reduced dimensionality scenario. However, the magnitude of improvement is notably smaller, which can be attributed to several factors. The first factor is a smaller number of function evaluations, as it could not properly apply parameter optimization using PPO enhancement. We theorize that iSOMA-RL should perform better the more complex the problem is. Secondary the problem could be less complex and such do not need parameter optimization that much.

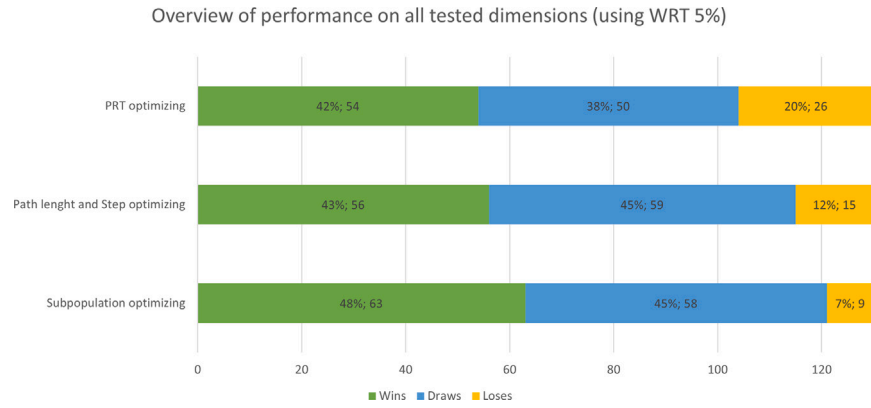


Fig. 10. Total number of wins, losses and draws in both tested dimensions.

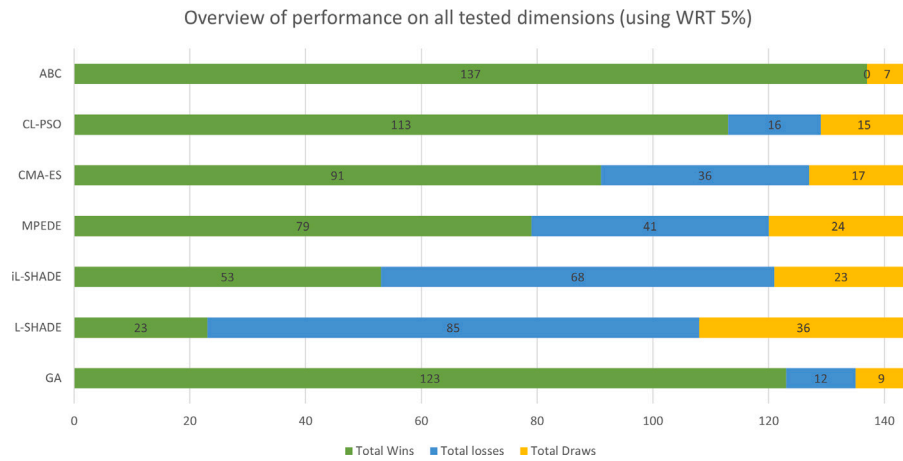


Fig. 11. Total number of wins, losses and draws in both tested dimensions (Win means that iSOMA-RL performed better).

Table 16

Algorithm Performance Comparison in 10 dimensions (Win means that iSOMA-RL performed better).

Algorithm	iSOMA-RL		iSOMA		Improvement	
	Win	Loss	Win	Loss	Win	Loss
ABC	67	0	66	0	+1	0
CL PSO	54	7	49	11	+5	-4
CMA-ES	45	14	43	13	+2	-1
L-Shade	9	41	8	46	+1	-5
iL-Shade	27	32	15	48	+2	-4
MPEDE	45	12	43	18	+2	-6
GA	61	3	60	6	+1	-3

Interestingly, the table also highlights iSOMA-RL's commendable performance against the formidable L-Shade algorithm, showcasing its robustness even in lower-dimensional spaces. The results further underscore the relative effectiveness of the algorithms, with ABC, CL PSO, CMA-ES, iL-SHADE, MPEDE, and GA demonstrating varying degrees of performance.

All results in terms of mean and standard deviation can be found in supplemental file.

5.2.1. Discussion of results against other algorithms

The comparative analysis of various optimization algorithms in both 30-dimensional and 10-dimensional scenarios yields intriguing insights. In the higher-dimensional setting, iSOMA-RL consistently outperforms its predecessor iSOMA across all evaluated scenarios, showcasing its enhanced capability in tackling complex optimization problems. This is particularly striking considering iSOMA's reputation for robustness,

as demonstrated by its formidable performance against the challenging L-Shade algorithm.

Furthermore, the comprehensive defeat of the ABC algorithm by iSOMA-RL underscores its superior effectiveness in optimizing the given set of functions. While other algorithms, such as CL PSO, CMA-ES, and GA, demonstrate varying degrees of performance, iSOMA-RL consistently proves to be the more effective choice. In the reduced 10-dimensional context, the superiority of iSOMA-RL over iSOMA remains evident, albeit with a more modest improvement. This can be attributed to a smaller number of test functions and constrained function evaluations, which limit the potential for substantial gains. Nevertheless, iSOMA-RL's ability to outperform even the formidable L-Shade algorithm in this context underscores its robustness and potential for optimization tasks across diverse computational environments. Overall, these findings strongly support the adoption of iSOMA-RL as a powerful optimization tool, especially in scenarios where iSOMA traditionally excels, but also in cases where it faces more formidable challenges.

We theorize that an increase in the complexity of functions and the number of function evaluations enhances the performance of iSOMA-RL, addressing the limitations of iSOMA, particularly in highly complex problems with high dimensionality.

5.3. Evaluation of the effectiveness of the proposed methods

Overall, all variants of the iSOMA-RL algorithm showed good performance when applied to the optimization problem at hand and can be used as an enhancement of the iSOMA algorithm as can be seen in Fig. 10, where we can see that subpopulation optimizing variant is most promising and number of losses is small compared to wins (see Fig. 11)

The performance of the proposed approach consistently outperformed other algorithms, including iSOMA, across all tested scenarios. This improvement was particularly noteworthy when compared to L-SHADE, where the original iSOMA algorithm performed less favorably on the specified CEC benchmarks. This effect was more pronounced in the 30-dimensional tests, where the addition of PPO enabled the optimization of parameters over an extended period, allowing for greater adaptability to the given function. This becomes increasingly challenging in lower dimensions due to the constrained number of function evaluations and reduced problem complexity.

It is important to note that the results should be taken in the context of the specific problem and the specific parameters used, as different problems and different parameters may lead to different results. It is important to conduct further experiments and analyses to determine the best variant for a specific problem and setting. Interestingly, when the problem is simpler than iSOMA-RL performs worse than when the problem is more complex. This could indicate that reinforcement learning for simpler problems is too complex or does not have enough time to learn properly.

5.4. Discussion of L-SHADE variants and iSOMA-RL

The L-SHADE algorithm exhibited superior performance compared to the iSOMA-RL, as evidenced by the original iSOMA algorithm's results. Although iSOMA-RL contributed to improved outcomes in 18 cases, it appears that the foundational model of iSOMA may be less effective on the benchmarks utilized in this study. This observation indicates that the incorporation of reinforcement learning enhancements can significantly boost iSOMA's performance.

The L-SHADE algorithm's apparent superiority over iSOMA-RL, as evidenced in our study, becomes more nuanced when considering recent variants like iL-SHADE and MPEDE. Despite claims of enhancement over L-SHADE, these variants, in our tests, did not surpass L-SHADE's performance. This discrepancy might be partially attributed to the parameter settings used, which were based on recommendations from their respective papers. It is possible that these settings, while theoretically sound, may not align perfectly with the specific demands of our test scenarios. Therefore, the observed underperformance of iL-SHADE and MPEDE could be more reflective of the sensitive nature of parameter configurations rather than a fundamental flaw in the algorithms themselves.

5.4.1. Comparison success-history-based parameter adaptation and reinforcement learning

The SHADE [29] operates by adapting its parameters based on historical success data. This method leverages past performance metrics to fine-tune the evolutionary process. It constantly updates a set of evolving parameter sets, adjusting them according to their effectiveness in generating successful outcomes. This approach is particularly useful in scenarios where past experiences offer valuable insights into optimal parameter settings. On the other hand, our reinforcement learning-based strategy employs a learning mechanism that rewards parameter configurations based on their optimization performance. This dynamic learning process is akin to an agent interacting with and learning from its environment, offering a more autonomous adjustment of parameters.

In terms of practical applications, L-SHADE [78,79] has shown effectiveness in a variety of optimization problems, especially where historical data is a significant indicator of future performance. Its strength lies in its ability to use past experiences to guide future parameter adjustments, making it robust in familiar optimization contexts. Conversely, the reinforcement learning approach is adaptable in environments where historical data might not be as informative. This method is advantageous in situations that require real-time parameter tuning or in problems where the optimization landscape is subject to change.

Table 17

Comparison of wins, losses, and draws for different variants against iSOMA-RL with Complex Reward 2 with statistical significance (WRT 5%), 10 dimensions.

Reward function	Better	Worse	Draws	Total
Simple	0	0	28	28
Difference	2	0	26	28
No penalty	2	0	26	28
Complex reward	0	0	28	28

By continuously adjusting strategy parameters like crossover rate and scaling factor based on past successes, SHADE can potentially escape local minima by exploring new regions of the search space. However, its performance in this aspect can depend heavily on the nature of the problem and the landscape of the optimization function.

On the other hand, PPO [19], a reinforcement learning algorithm, is designed to optimize policies in a way that balances exploration and exploitation. PPO aims to take the biggest possible improvement step on policy while avoiding large updates that could collapse performance. This characteristic of PPO can be beneficial in avoiding local minima in policy optimization problems. PPO's approach of clipping the policy update function helps in maintaining a controlled exploration of the policy space, which can prevent the algorithm from getting stuck in local minima.

5.4.2. The need for more research

The comparative analysis of success-history adaptation and PPO offers a fertile ground for scholarly investigation, particularly in their efficacy in circumventing local minima within optimization processes. The theoretical underpinnings and pragmatic applications of these algorithms indicate inherent strengths in navigating intricate optimization landscapes. Nonetheless, it is imperative to undertake empirical research to thoroughly ascertain and quantify their performance across diverse contexts. A focal area of interest should be the amalgamation of adaptive parameter calibration with PPO's sophisticated policy optimization technique. This synthesis could potentially unveil synergistic effects that significantly bolster the algorithms' capability to elude local minima. Such investigative efforts would not only deepen the academic understanding of these algorithms but might also pave the way for the emergence of more robust and efficient optimization methodologies.

Moreover, it is crucial to evaluate and experiment with various parameter adaptation strategies, such as Q-learning, PPO, and Success-history-based adaptation, across different algorithms to gauge their efficacy and identify any inherent limitations or challenges.

5.5. Comparison of rewards function

We compare our rewards function with Complex Reward 2, which was used in previous benchmarks carried out in this study.

Table 17 compares the performance of different variants of an algorithm using different reward functions. The variants are evaluated using four different reward functions to the Complex Reward 2 function: Simple, Difference, No penalty, and Complex reward. The results are presented in terms of wins, losses, and draws.

It is clear that all variants performed similarly across all four reward functions. It is important to note that these results are specific to the 10-dimensional case and may not necessarily apply to other-dimensional cases. Additionally, it is also important to mention that although the table shows some variants performing better than others, it is important to note that all variants performed well and are suitable for use depending on the specific problem and constraints.

Table 18

Sample results of time consumption for optimizing functions with and without reinforcement learning (30 dimensions).

Function	iSOMA-RL (ms)	iSOMA (ms)	Difference (ms)
Rotated Schwefel's F.	95.5	65.6	29.9
Rotated High Conditioned Elliptic F.	62.5	43.3	19.3
Composition Function 2 (N=3)	110.6	97.3	13.3
Hybrid F. 10	296.6	283.3	13.3
Hybrid F. 2	132.1	120.5	11.7

Table 19

Sample results of time consumption for optimizing functions with and without reinforcement learning (10 dimensions).

Function	iSOMA-RL (ms)	iSOMA (ms)	Difference (ms)
Sphere F.	5.3	5.8	0.5
Rotated Schwefel's F.	18.8	15.6	3.2
Non-continuous Rotated Rastrigin's F.	19.1	15.1	4.0
Schwefel's F.	19.2	15.3	3.9

5.6. Results for overhead of using reinforcement learning

Our benchmarking analysis revealed a relatively constant (for given function evaluations) overhead when using reinforcement learning with iSOMA-RL. Specifically, we found a average 17.4 ms (14%) overhead for functions with 30 dimensions and a average 3.6 ms (28%) overhead for functions with 10 dimensions. It is important to note that the level of complexity of the benchmark function affected the practicality and usefulness of applying iSOMA-RL. As such, for more complex benchmark functions, the use of iSOMA-RL with reinforcement learning may be a more practical and viable option. It is likely that increasing the number of function evaluations will result in a higher overhead when using reinforcement learning. However, it should be emphasized that the observed difference in overhead is relatively small in comparison to the time required for evaluating the function being optimized. It is worth noting that PPO can be executed on a GPU, although we did not evaluate its performance in this context.

Table 18 presents a subset of the results obtained from benchmarking the functions with 30 dimensions, while Table 19 displays a subset of the results obtained for benchmark functions with 10 dimensions. It should be noted that the observed differences in the algorithmic performance between these two sets of benchmarks can be attributed to the relative difference in the complexity of the functions being tested. Specifically, functions with 10 dimensions exhibit lower complexity, resulting in faster algorithms compared to those with 30 dimensions.

6. Conclusion

In this study, several variants of the iSOMA algorithm were proposed and compared with the original iSOMA algorithm. The performance of the proposed variants was evaluated by comparing the results with the original iSOMA algorithm using a set of benchmark functions in 30 (Table 11) and 10 (Table 14) dimensions on three different CEC benchmarks (2013, 2015, 2017).

The subpopulation optimization variant was found to be the best performing variant among the proposed variants and showed a significant improvement over the original iSOMA algorithm in both 30 and 10 dimensions. For 30 dimensions it was in more than 50% (39 wins over 33 draws and losses) cases better than the original iSOMA, which is a significant improvement for iSOMA algorithm.

We conducted a comparative analysis of the subpopulation optimization variant against several widely employed algorithms. Our findings consistently demonstrated improvements against over the original iSOMA across all tested algorithms (L-SHADE, ABC, CL PSO, GA, CMA-ES). The primary impact was observed in 30 dimensions, with

a lesser yet still noticeable effect in 10 dimensions. This leads us to theorize that iSOMA-RL may be particularly well-suited for addressing more complex problem domains, where the adaptive parameter tuning facilitated by Proximal Policy Optimization (PPO) can be especially advantageous.

We had found that using reinforcement learning for SOMA family function can be beneficial, and more research can be done this way. It will be interesting if performance increases when we pre-train algorithm. This way algorithm can learn from its experience and try more optimal way to optimize functions. Also, an interesting question is whether a combination of more parameters will be more effective when pre-training is used. Currently, we believe that having more parameters can hinder optimal learning when used on a single problem as it cannot explore the influence of parameters without pre-training. The open question is also whether any other RL algorithm like SAC, A3C or else could perform better than PPO.

We also benchmarked reward functions and found that there was no significant difference between them as can be seen in a Table 17, and the reward functions used in the literature were found pretty equal in the case of iSOMA-RL, when PPO is used. Furthermore, our findings suggest that the inclusion of reinforcement learning does not significantly impede the computational efficiency of the algorithm.

CRediT authorship contribution statement

Lukáš Klein: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Ivan Zelinka:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Formal analysis, Conceptualization. **David Seidl:** Supervision, Resources, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

I would like to acknowledge the support provided by the grant SGS No. SP2023/050 from VSB-Technical University of Ostrava and TN02000025 National Centre for Energy II, which made this research possible. This article has been also produced with the support of the European Union under the REFRESH–Research Excellence For Region Sustainability and High-tech Industries project number CZ.10.03.01/00/22-003/0000048 via the Operational Programme Just Transition.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.swevo.2024.101487>.

References

- [1] A. Darwish, Bio-inspired computing: Algorithms review, deep analysis, and the scope of applications, *Future Comput. Inform. J.* 3 (2) (2018) 231–246.
- [2] K.V. Price, Differential evolution, in: I. Zelinka, V. Šnášel, A. Abraham (Eds.), *Handbook of Optimization: From Classical To Modern Approach*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 187–214, <http://dx.doi.org/10.1007/978-3-642-30504-7y8>.
- [3] M. Dorigo, T. Stützle, *Ant colony optimization: overview and recent advances*, *Handb. Metaheuristics* (2019) 311–351.
- [4] D. Karaboga, B. Gorkemli, C. Ozturk, N. Karaboga, A comprehensive survey: artificial bee colony (ABC) algorithm and applications, *Artif. Intell. Rev.* 42 (1) (2014) 21–57.
- [5] J. Kennedy, R. Eberhart, Particle swarm optimization, in: *Proceedings of ICNN'95-International Conference on Neural Networks*, Vol. 4, IEEE, 1995, pp. 1942–1948.
- [6] I. Zelinka, SOMA—self-organizing migrating algorithm, in: *Self-Organizing Migrating Algorithm*, Springer, 2016, pp. 3–49.
- [7] I. Zelinka, SOMA — Self-organizing migrating algorithm, in: *New Optimization Techniques in Engineering*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 167–217, <http://dx.doi.org/10.1007/978-3-540-39930-8y7>.
- [8] L. Skanderova, T. Fabian, I. Zelinka, Self-adapting self-organizing migrating algorithm, *Swarm Evol. Comput.* 51 (2019) 100593, <http://dx.doi.org/10.1016/j.swevo.2019.100593>, URL <https://www.sciencedirect.com/science/article/pii/S2210650219300756>.
- [9] T. Kadavy, M. Pluhacek, A. Viktorin, R. Senkerik, SOMA-CLP for competition on bound constrained single objective numerical optimization benchmark: A competition entry on bound constrained single objective numerical optimization at the genetic and evolutionary computation conference (GECCO) 2021, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO '21*, Association for Computing Machinery, New York, NY, USA, 2021, pp. 11–12, <http://dx.doi.org/10.1145/3449726.3463286>.
- [10] H. Ding, Y. Gu, H. Wu, J. Zhou, NL-SOMA-CLP for real parameter single objective bound constrained optimization, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO '22*, Association for Computing Machinery, New York, NY, USA, 2022, pp. 5–6, <http://dx.doi.org/10.1145/3520304.3534051>.
- [11] D. Singh, S. Agrawal, Self organizing migrating algorithm with quadratic interpolation for solving large scale global optimization problems, *Appl. Soft Comput.* 38 (2016) 1040–1048, <http://dx.doi.org/10.1016/j.asoc.2015.09.033>, URL <https://www.sciencedirect.com/science/article/pii/S1568494615006055>.
- [12] Q.B. Diep, T.C. Truong, S. Das, I. Zelinka, Self-organizing migrating algorithm with narrowing search space strategy for robot path planning, *Appl. Soft Comput.* 116 (2022) 108270, <http://dx.doi.org/10.1016/j.asoc.2021.108270>, URL <https://www.sciencedirect.com/science/article/pii/S1568494621010851>.
- [13] Q.B. Diep, Self-organizing migrating algorithm team to team adaptive – SOMA T3A, in: 2019 IEEE Congress on Evolutionary Computation, CEC, 2019, pp. 1182–1187, <http://dx.doi.org/10.1109/CEC.2019.8790202>.
- [14] Q.B. Diep, I. Zelinka, S. Das, R. Senkerik, SOMA T3A for solving the 100-digit challenge, in: A. Zamuda, S. Das, P.N. Suganthan, B.K. Panigrahi (Eds.), *Swarm, Evolutionary, and Memetic Computing and Fuzzy and Neural Computing*, Springer International Publishing, Cham, 2020, pp. 155–165.
- [15] Q.B. Diep, I. Zelinka, S. Das, Self-organizing migrating algorithm Pareto, *MENDEL* 25 (1) (2019) 111–120, <http://dx.doi.org/10.13164/mendel.2019.1.111>.
- [16] T.C. Truong, Q.B. Diep, I. Zelinka, R. Senkerik, Pareto-based self-organizing migrating algorithm solving 100-digit challenge, in: A. Zamuda, S. Das, P.N. Suganthan, B.K. Panigrahi (Eds.), *Swarm, Evolutionary, and Memetic Computing and Fuzzy and Neural Computing*, Springer International Publishing, Cham, 2020, pp. 13–20.
- [17] K. Price, N. Awad, M. Ali, P. Suganthan, The 2019 100-Digit Challenge on Real-Parameter, Single Objective Optimization: Analysis of Results, Rep 2019, 2019, Dept. Computer Science, University of Freiburg, Freiburg, Germany, School of Computer Information Systems, Jordan University of Science and Technology, Jordan, and School of EEE, Nanyang Technol. Univ., Singapore, 2019.
- [18] L.P. Kaelbling, M.L. Littman, A.W. Moore, Reinforcement learning: A survey, *J. Artificial Intelligence Res.* 4 (1996) 237–285.
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, <http://dx.doi.org/10.48550/ARXIV.1707.06347>, URL <https://arxiv.org/abs/1707.06347>.
- [20] E. Bohn, E.M. Coates, S. Moe, T.A. Johansen, Deep reinforcement learning attitude control of fixed-wing uavs using proximal policy optimization, in: 2019 International Conference on Unmanned Aircraft Systems, ICUAS, IEEE, 2019, pp. 523–533.
- [21] F. Ye, X. Cheng, P. Wang, C.-Y. Chan, J. Zhang, Automated lane change strategy using proximal policy optimization-based deep reinforcement learning, in: 2020 IEEE Intelligent Vehicles Symposium, IV, IEEE, 2020, pp. 1746–1752.
- [22] L.C. Melo, D.C. Melo, M.R. Maximo, Learning humanoid robot running motions with symmetry incentive through proximal policy optimization, *J. Intell. Robot. Syst.* 102 (3) (2021) 1–15.
- [23] Y. Xu, D. Pi, A reinforcement learning-based communication topology in particle swarm optimization, *Neural Comput. Appl.* 32 (14) (2019) 10007–10032, <http://dx.doi.org/10.1007/s00521-019-04527-9>.
- [24] C. Qu, W. Gai, M. Zhong, J. Zhang, A novel reinforcement learning based grey wolf optimizer algorithm for unmanned aerial vehicles (UAVs) path planning, *Appl. Soft Comput.* 89 (2020) 106099, <http://dx.doi.org/10.1016/j.asoc.2020.106099>, URL <https://www.sciencedirect.com/science/article/pii/S1568494620300399>.
- [25] G. Karafotias, A.E. Eiben, M. Hoogendoorn, Generic parameter control with reinforcement learning, in: *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, GECCO '14*, Association for Computing Machinery, New York, NY, USA, 2014, pp. 1319–1326, <http://dx.doi.org/10.1145/2576768.2598360>.
- [26] E. Emary, H.M. Zawbaa, C. Grosan, Experienced gray wolf optimization through reinforcement learning and neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* 29 (3) (2018) 681–694, <http://dx.doi.org/10.1109/TNNLS.2016.2634548>.
- [27] G.S. Piperagkas, G. Georgoulas, K.E. Parsopoulos, C.D. Stylios, A.C. Likas, Integrating particle swarm optimization with reinforcement learning in noisy problems, in: *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation*, ACM, 2012, <http://dx.doi.org/10.1145/2330163.2330173>.
- [28] I. Zelinka, SOMA—Self-organizing migrating algorithm, in: D. Davendra, I. Zelinka (Eds.), *Self-Organizing Migrating Algorithm: Methodology and Implementation*, Springer International Publishing, Cham, 2016, pp. 3–49, <http://dx.doi.org/10.1007/978-3-319-28161-2y1>.
- [29] R. Tanabe, A. Fukunaga, Evaluating the performance of SHADE on CEC 2013 benchmark problems, in: 2013 IEEE Congress on Evolutionary Computation, 2013, pp. 1952–1959, <http://dx.doi.org/10.1109/CEC.2013.6557798>.
- [30] N. Awad, M.Z. Ali, R.G. Reynolds, A differential evolution algorithm with success-based parameter adaptation for CEC2015 learning-based optimization, in: 2015 IEEE Congress on Evolutionary Computation, CEC, 2015, pp. 1098–1105, <http://dx.doi.org/10.1109/CEC.2015.7257012>.
- [31] Y.-J. Zheng, X.-B. Wu, Tuning maturity model of ecogeography-based optimization on CEC 2015 single-objective optimization test problems, in: 2015 IEEE Congress on Evolutionary Computation, CEC, 2015, pp. 1018–1024, <http://dx.doi.org/10.1109/CEC.2015.7257001>.
- [32] J.J. Liang, L. Guo, R. Liu, B.Y. Qu, A self-adaptive dynamic particle swarm optimizer, in: 2015 IEEE Congress on Evolutionary Computation, CEC, 2015, pp. 3206–3213, <http://dx.doi.org/10.1109/CEC.2015.7257290>.
- [33] D. Jagodziński, J. Arabas, A differential evolution strategy, in: 2017 IEEE Congress on Evolutionary Computation, CEC, 2017, pp. 1872–1876, <http://dx.doi.org/10.1109/CEC.2017.7969529>.
- [34] A. Tangherloni, L. Rundo, M.S. Nobile, Proactive particles in swarm optimization: A settings-free algorithm for real-parameter single objective optimization problems, in: 2017 IEEE Congress on Evolutionary Computation, CEC, 2017, pp. 1940–1947, <http://dx.doi.org/10.1109/CEC.2017.7969538>.
- [35] R. Kommadath, P. Kotecha, Teaching learning based optimization with focused learning and its performance on CEC2017 functions, in: 2017 IEEE Congress on Evolutionary Computation, CEC, 2017, pp. 2397–2403, <http://dx.doi.org/10.1109/CEC.2017.7969595>.
- [36] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T.P. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, 2016, <http://dx.doi.org/10.48550/ARXIV.1602.01783>, URL <https://arxiv.org/abs/1602.01783>.
- [37] E. Akanksha, N. Sharma, K. Gulati, et al., Review on reinforcement learning, research evolution and scope of application, in: 2021 5th International Conference on Computing Methodologies and Communication, ICCMC, IEEE, 2021, pp. 1416–1423.
- [38] R. Nian, J. Liu, B. Huang, A review on reinforcement learning: Introduction and applications in industrial process control, *Comput. Chem. Eng.* 139 (2020) 106886.
- [39] A. Perera, P. Kamalaruban, Applications of reinforcement learning in energy systems, *Renew. Sustain. Energy Rev.* 137 (2021) 110618.
- [40] K. Nussenbaum, C.A. Hartley, Reinforcement learning across development: What insights can we draw from a decade of research? *Dev. Cogn. Neurosci.* 40 (2019) 100733.
- [41] C.J. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (3) (1992) 279–292.
- [42] R.J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Mach. Learn.* 8 (3–4) (1992) 229–256, <http://dx.doi.org/10.1007/bf00992696>.
- [43] K. Arulkumaran, M.P. Deisenroth, M. Brundage, A.A. Bharath, Deep reinforcement learning: A brief survey, *IEEE Signal Process. Mag.* 34 (6) (2017) 26–38, <http://dx.doi.org/10.1109/MSP.2017.2743240>.
- [44] R. Nian, J. Liu, B. Huang, A review on reinforcement learning: Introduction and applications in industrial process control, *Comput. Chem. Eng.* 139 (2020) 106886, <http://dx.doi.org/10.1016/j.compchemeng.2020.106886>, URL <https://www.sciencedirect.com/science/article/pii/S0098135420300557>.
- [45] Y. Yu, Towards sample efficient reinforcement learning, in: *IJCAI*, 2018, pp. 5739–5743.

- [46] J.C.V. Tieck, M.V. Pogančić, J. Kaiser, A. Roennau, M.-O. Gewaltig, R. Dillmann, Learning continuous muscle control for a multi-joint arm by extending proximal policy optimization with a liquid state machine, in: *International Conference on Artificial Neural Networks*, Springer, 2018, pp. 211–221.
- [47] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, L. Zhang, Solving Rubik's cube with a robot hand, 2019, <http://dx.doi.org/10.48550/ARXIV.1910.07113>, URL <https://arxiv.org/abs/1910.07113>.
- [48] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. Pinto, J. Raiman, T. Salimans, J. Schlatter, S. Zhang, Dota 2 with large scale deep reinforcement learning, 2019.
- [49] G. Libardi, G. De Fabritiis, S. Dittert, Guided exploration with proximal policy optimization using a single demonstration, in: M. Meila, T. Zhang (Eds.), *Proceedings of the 38th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 139, PMLR, 2021, pp. 6611–6620, URL <https://proceedings.mlr.press/v139/libardi21a.html>.
- [50] M. August, J.M. Hernández-Lobato, Taking gradients through experiments: LSTMs and memory proximal policy optimization for black-box quantum control, in: R. Yokota, M. Weiland, J. Shalf, S. Alam (Eds.), *High Performance Computing*, Springer International Publishing, Cham, 2018, pp. 591–613.
- [51] H. Nobakht, Y. Liu, Action space noise optimization as exploration in deterministic policy gradient for locomotion tasks, *Appl. Intell.* 52 (12) (2022) 14218–14232, <http://dx.doi.org/10.1007/s10489-021-02995-y>.
- [52] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, *Adv. Eng. Softw.* 69 (2014) 46–61.
- [53] W. Long, J. Jiao, X. Liang, M. Tang, An exploration-enhanced grey wolf optimizer to solve high-dimensional numerical optimization, *Eng. Appl. Artif. Intell.* 68 (2018) 63–80, <http://dx.doi.org/10.1016/j.engappai.2017.10.024>, URL <https://www.sciencedirect.com/science/article/pii/S0952197617302713>.
- [54] Y. Song, L. Wei, Q. Yang, J. Wu, L. Xing, Y. Chen, RL-GA: A reinforcement learning-based genetic algorithm for electromagnetic detection satellite scheduling problem, *Swarm Evol. Comput.* 77 (2023) 101236, <http://dx.doi.org/10.1016/j.swevo.2023.101236>, URL <https://www.sciencedirect.com/science/article/pii/S221065022300010X>.
- [55] S. Yin, M. Jin, H. Lu, G. Gong, W. Mao, G. Chen, W. Li, Reinforcement-learning-based parameter adaptation method for particle swarm optimization, *Complex Intell. Syst.* 9 (5) (2023) 5585–5609, <http://dx.doi.org/10.1007/s40747-023-01012-8>, URL <http://dx.doi.org/10.1007/s40747-023-01012-8>.
- [56] W. Huang, Y. Liu, X. Zhang, Hybrid particle swarm optimization algorithm based on the theory of reinforcement learning in psychology, *Systems* 11 (2) (2023) <http://dx.doi.org/10.3390/systems11020083>, URL <https://www.mdpi.com/2079-8954/11/2/83>.
- [57] H. Basak, M. Das, S. Modak, RSO: A novel reinforced swarm optimization algorithm for feature selection, 2021, <http://dx.doi.org/10.48550/ARXIV.2107.14199>, URL <https://arxiv.org/abs/2107.14199>.
- [58] F. Wang, X. Wang, S. Sun, A reinforcement learning level-based particle swarm optimization algorithm for large-scale optimization, *Inform. Sci.* 602 (2022) 298–312, <http://dx.doi.org/10.1016/j.ins.2022.04.053>, URL <https://www.sciencedirect.com/science/article/pii/S0020025522004054>.
- [59] H. Iima, Y. Kuroe, Swarm reinforcement learning algorithm based on particle swarm optimization whose personal bests have lifespans, in: C.S. Leung, M. Lee, J.H. Chan (Eds.), *Neural Information Processing*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 169–178.
- [60] S. Yin, Y. Liu, G. Gong, H. Lu, W. Li, RLEPSO: Reinforcement learning based ensemble particle swarm optimizer, in: 2021 4th International Conference on Algorithms, Computing and Artificial Intelligence, ACM, 2021, <http://dx.doi.org/10.1145/3508546.3508599>.
- [61] L. Lu, H. Zheng, J. Jie, M. Zhang, R. Dai, Reinforcement learning-based particle swarm optimization for sewage treatment control, *Complex Intell. Syst.* 7 (5) (2021) 2199–2210, <http://dx.doi.org/10.1007/s40747-021-00395-w>.
- [62] Y. Liu, H. Lu, S. Cheng, Y. Shi, An adaptive online parameter control algorithm for particle swarm optimization based on reinforcement learning, in: 2019 IEEE Congress on Evolutionary Computation, CEC, 2019, pp. 815–822, <http://dx.doi.org/10.1109/CEC.2019.8790035>.
- [63] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, 2015, <http://dx.doi.org/10.48550/ARXIV.1509.02971>, URL <https://arxiv.org/abs/1509.02971>.
- [64] Q. Liao, Y. Guo, Y. Tu, H. Zhang, Fidelity-based ant colony algorithm with Q-learning of quantum system, *Internat. J. Theoret. Phys.* 57 (3) (2017) 862–876, <http://dx.doi.org/10.1007/s10773-017-3619-9>.
- [65] G. Karafotias, A.E. Eiben, M. Hoogendoorn, Generic parameter control with reinforcement learning, in: *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, ACM, 2014, <http://dx.doi.org/10.1145/2576768.2598360>.
- [66] D. Zhao, H. Wang, K. Shao, Y. Zhu, Deep reinforcement learning with experience replay based on SARSA, in: 2016 IEEE Symposium Series on Computational Intelligence, SSCI, 2016, pp. 1–6, <http://dx.doi.org/10.1109/SSCI.2016.7849837>.
- [67] W. Li, P. Liang, B. Sun, Y. Sun, Y. Huang, Reinforcement learning-based particle swarm optimization with neighborhood differential mutation strategy, *Swarm Evol. Comput.* 78 (2023) 101274, <http://dx.doi.org/10.1016/j.swevo.2023.101274>, URL <https://www.sciencedirect.com/science/article/pii/S2210650223000482>.
- [68] A. Seyyedabbasi, A reinforcement learning-based metaheuristic algorithm for solving global optimization problems, *Adv. Eng. Softw.* 178 (2023) 103411, <http://dx.doi.org/10.1016/j.advengsoft.2023.103411>, URL <https://www.sciencedirect.com/science/article/pii/S0965997823000030>.
- [69] L.M. Gambardella, M. Dorigo, Ant-Q: A reinforcement learning approach to the traveling salesman problem, 1995, pp. 252–260.
- [70] M.I. Radaideh, K. Shirvan, PESA: Prioritized experience replay for parallel hybrid evolutionary and swarm algorithms - Application to nuclear fuel, *Nucl. Eng. Technol.* 54 (10) (2022) 3864–3877, <http://dx.doi.org/10.1016/j.net.2022.05.001>, URL <https://www.sciencedirect.com/science/article/pii/S1738573322002480>.
- [71] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-Baselines3: Reliable reinforcement learning implementations, *J. Mach. Learn. Res.* 22 (268) (2021) 1–8, URL <http://jmlr.org/papers/v22/0-1364.html>.
- [72] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, OpenAI Gym, 2016, <http://dx.doi.org/10.48550/ARXIV.1606.01540>, URL <https://arxiv.org/abs/1606.01540>.
- [73] J.J. Liang, B. Qu, P.N. Suganthan, A.G. Hernández-Díaz, Problem Definitions and Evaluation Criteria for the CEC 2013 Special Session on Real-Parameter Optimization, Vol. 201212, No. 34, Technical Report, Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou, China and Nanyang Technological University, Singapore, 2013, pp. 281–295.
- [74] Q. Chen, B. Liu, Q.F. Zhang, J.J. Liang, P.N. Suganthan, B. Qu, Problem definitions and evaluation criteria for CEC 2015 special session on bound constrained single-objective computationally expensive numerical optimization, 2015.
- [75] A. Mzanh, J. Liang, B. Qu, P. Suganthan, Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective bound constrained real-parameter numerical optimization, 2016.
- [76] T. Nguyen, A framework of Optimization Functions using Numpy (OpFuNu) for Optimization Problems, Zenodo, 2020, <http://dx.doi.org/10.5281/zenodo.3620960>.
- [77] J. Carrasco, S. García, M. Rueda, S. Das, F. Herrera, Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review, *Swarm Evol. Comput.* 54 (2020) 100665, <http://dx.doi.org/10.1016/j.swevo.2020.100665>, URL <https://www.sciencedirect.com/science/article/pii/S2210650219302639>.
- [78] A.P. Piotrowski, L-SHADE optimization algorithms with population-wide inertia, *Inform. Sci.* 468 (2018) 117–141, <http://dx.doi.org/10.1016/j.ins.2018.08.030>, URL <https://www.sciencedirect.com/science/article/pii/S0020025518306352>.
- [79] J. Brest, M.S. Maučec, B. Bošković, IL-SHADE: Improved L-SHADE algorithm for single objective real-parameter optimization, in: 2016 IEEE Congress on Evolutionary Computation, CEC, 2016, pp. 1188–1195, <http://dx.doi.org/10.1109/CEC.2016.7743922>.
- [80] X. Li, L. Wang, Q. Jiang, N. Li, Differential evolution algorithm with multi-population cooperation and multi-strategy integration, *Neurocomputing* 421 (2021) 285–302, <http://dx.doi.org/10.1016/j.neucom.2020.09.007>, URL <https://www.sciencedirect.com/science/article/pii/S09525231220314107>.
- [81] N. Hansen, The CMA evolution strategy: A tutorial, 2016, <http://dx.doi.org/10.48550/ARXIV.1604.00772>, URL <https://arxiv.org/abs/1604.00772>.
- [82] D. Karaboga, B. Akay, Artificial bee colony (ABC), harmony search and bees algorithms on numerical optimization, 2009.
- [83] J. Liang, A. Qin, P. Suganthan, S. Baskar, Comprehensive learning particle swarm optimizer for global optimization of multimodal functions, *IEEE Trans. Evol. Comput.* 10 (3) (2006) 281–295, <http://dx.doi.org/10.1109/TEVC.2005.857610>.
- [84] N. Van Thieu, S. Mirjalili, MEALPY: An open-source library for latest meta-heuristic algorithms in Python, *J. Syst. Archit.* (2023) <http://dx.doi.org/10.1016/j.jsysarc.2023.102871>.